

Chapter 14

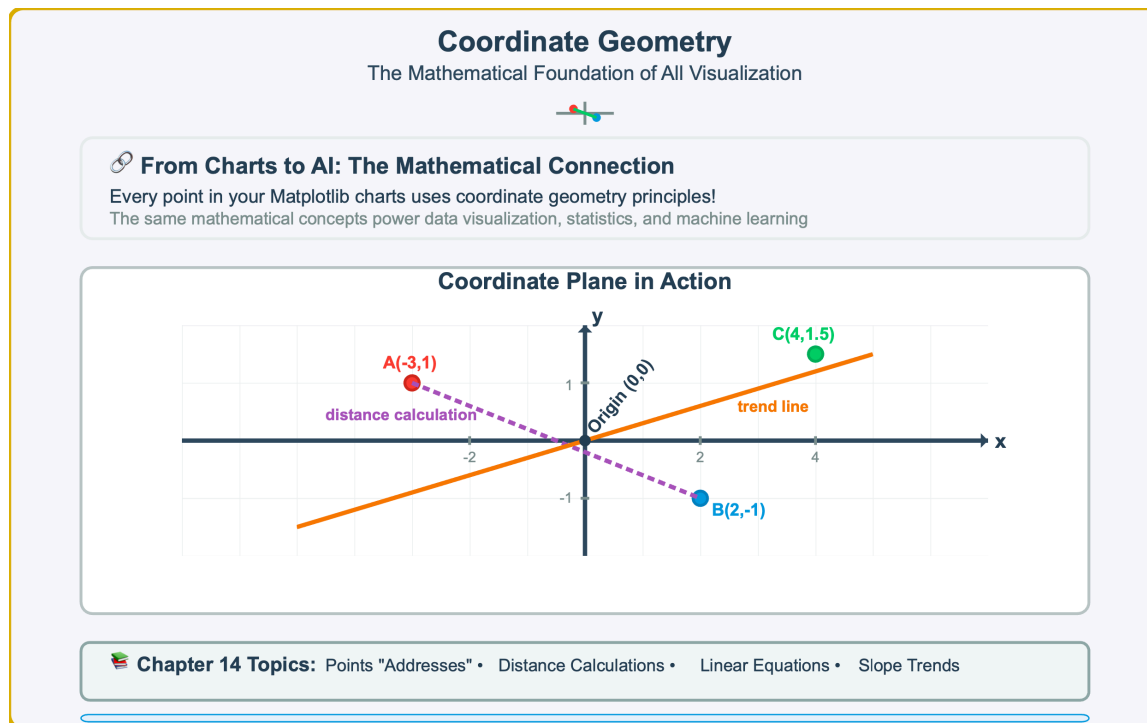


Figure 14.1 — Coordinate Geometry — The Mathematical Foundation of Visualization. This coordinate plane demonstrates how every point in data visualization uses mathematical coordinates, showing sample points A(2,3), B(5,2), and C(7,4) with distance calculations and trend lines that form the basis of all charts and machine learning algorithms.

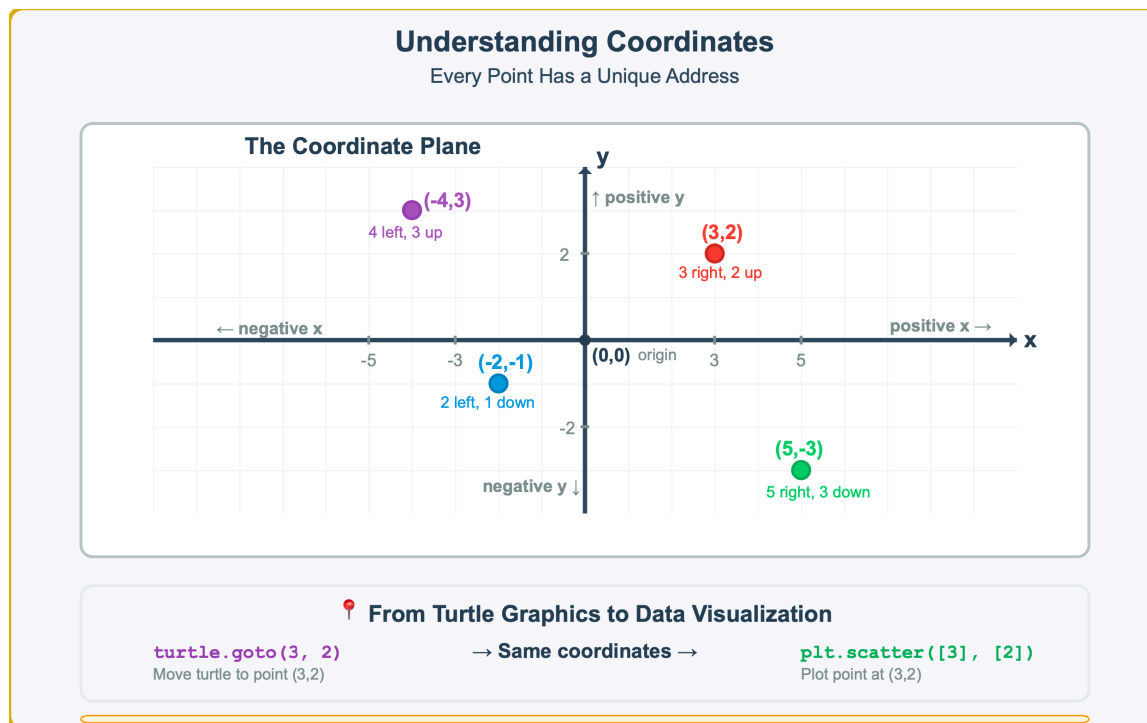


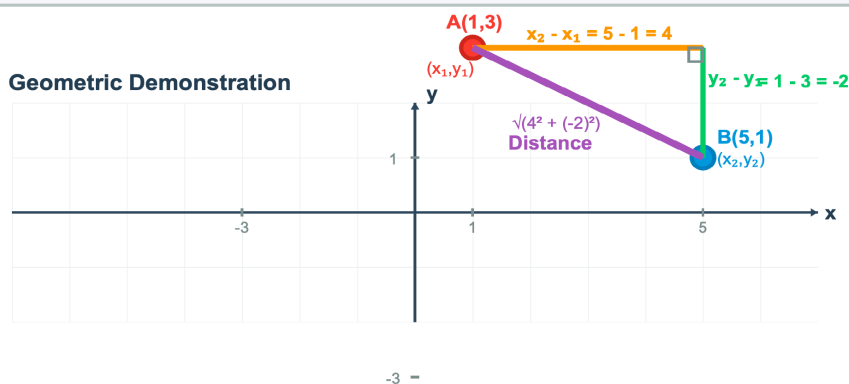
Figure 14.2 — Understanding Coordinates — Every Point Has an Address. This coordinate plane demonstrates how every point has a unique (x,y) address, showing examples like (3,2), (−2,−1), and (4,−2), with the origin at (0,0). The same coordinate principles apply from turtle graphics to Matplotlib plotting.

The Distance Formula

Euclidean Distance Using the Pythagorean Theorem

$$\text{Distance} = \sqrt{[(x_2 - x_1)^2 + (y_2 - y_1)^2]}$$

From Chapter 12: Measures how "similar" or "far apart" data points are



Step-by-Step Calculation:

$$\text{Distance} = \sqrt{[(5-1)^2 + (1-3)^2]} = \sqrt{[4^2 + (-2)^2]} = \sqrt{[16 + 4]} = \sqrt{20} \approx 4.47 \text{ units}$$

Figure 14.3 — The Distance Formula — Euclidean Distance. The distance between points A(1,3) and B(5,1) is calculated using the Pythagorean theorem: $\text{Distance} = \sqrt{[(x_2 - x_1)^2 + (y_2 - y_1)^2]} = \sqrt{[(5 - 1)^2 + (1 - 3)^2]} = \sqrt{20} \approx 4.47$ units, forming the foundation for data similarity measurements.

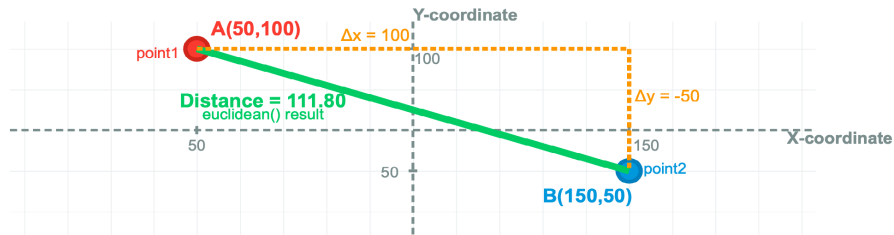
Distance Formula Implementation in Python

Using Matplotlib + SciPy for Mathematical Calculations

Python Implementation:

```
import matplotlib.pyplot as plt                    # Calculate distance
from scipy.spatial.distance import euclidean
distance = euclidean(point1, point2)
```

Distance Calculation Visualization



Mathematical Verification:

Distance = $\sqrt{[(150-50)^2 + (50-100)^2]} = \sqrt{[100^2 + (-50)^2]} = \sqrt{[10000 + 2500]} = \sqrt{12500} = 111.80$ units

Figure 14.4 — Distance Formula Implementation in Python. This example demonstrates calculating the distance between points A(50,100) and B(150,50) using Python's SciPy library, showing both the mathematical formula $\sqrt{[(150-50)^2 + (50-100)^2]} = 111.80$ and the corresponding code implementation with euclidean distance function.

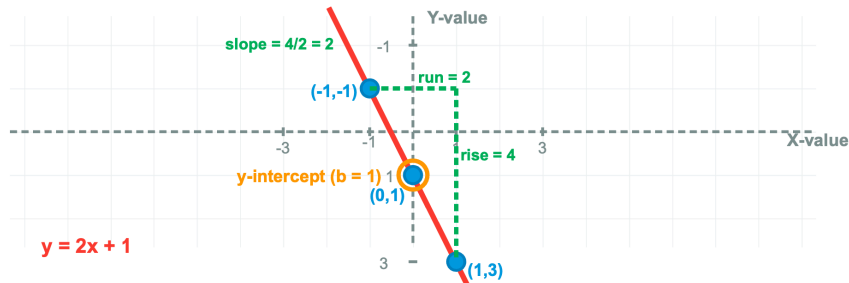
Linear Equation Implementation: $y = 2x + 1$

Programming a Straight Line in Python!

Python Implementation:

```
def linear_function(x):  
    return 2 * x + 1 # slope=2, intercept=1  
  
plt.plot(x_vals, y_vals,  
         color='red')
```

Linear Equation Visualization: $y = 2x + 1$



Linear Equation Components:

Slope (m) = 2: Line rises 2 units for every 1 unit right

Y-intercept (b) = 1: Line crosses y-axis at $y = 1$

Figure 14.5 — Linear Equation Implementation — $y = 2x + 1$. This example demonstrates programming a linear function in Python, showing how the equation $y = 2x + 1$ creates a straight line with slope $m = 2$ and y-intercept $b = 1$, plotted using matplotlib with points $(-1, -1)$, $(0, 1)$, and $(1, 3)$.

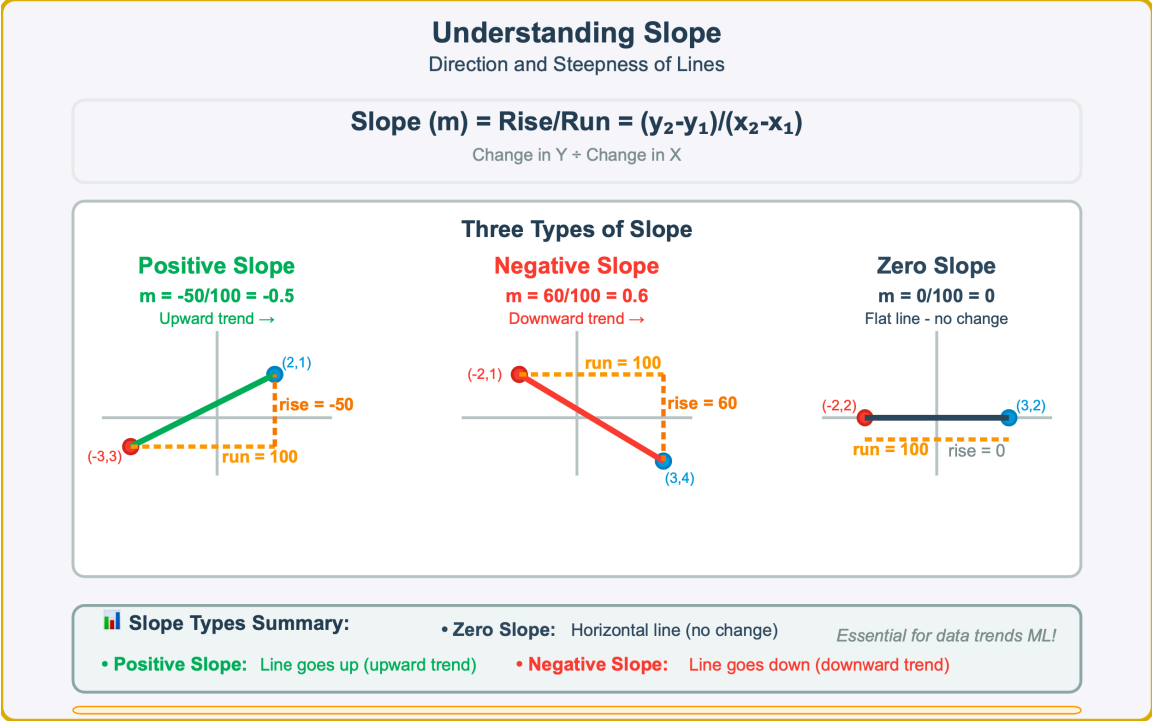
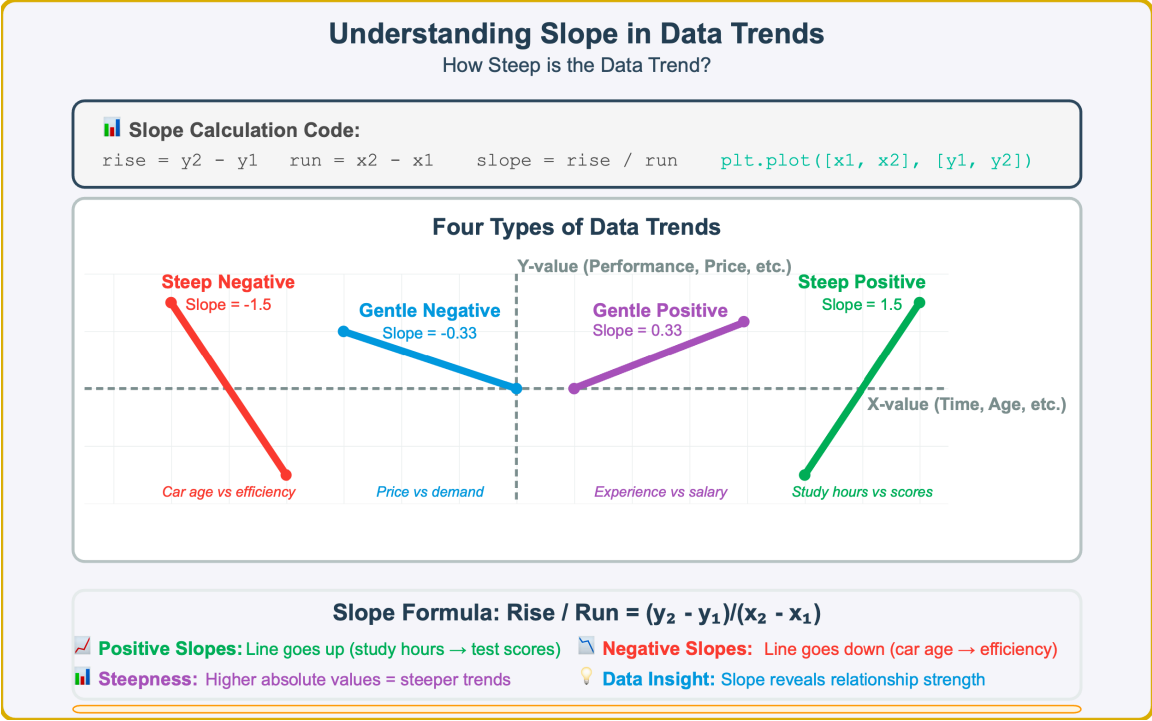


Figure 14.6 — Understanding Slope — Direction and Steepness of Lines. The slope formula (rise/run = $(y_2 - y_1) / (x_2 - x_1)$) demonstrates three types: positive slope (upward trend), negative slope (downward trend), and zero slope (flat line), fundamental concepts used in data trends and machine learning.

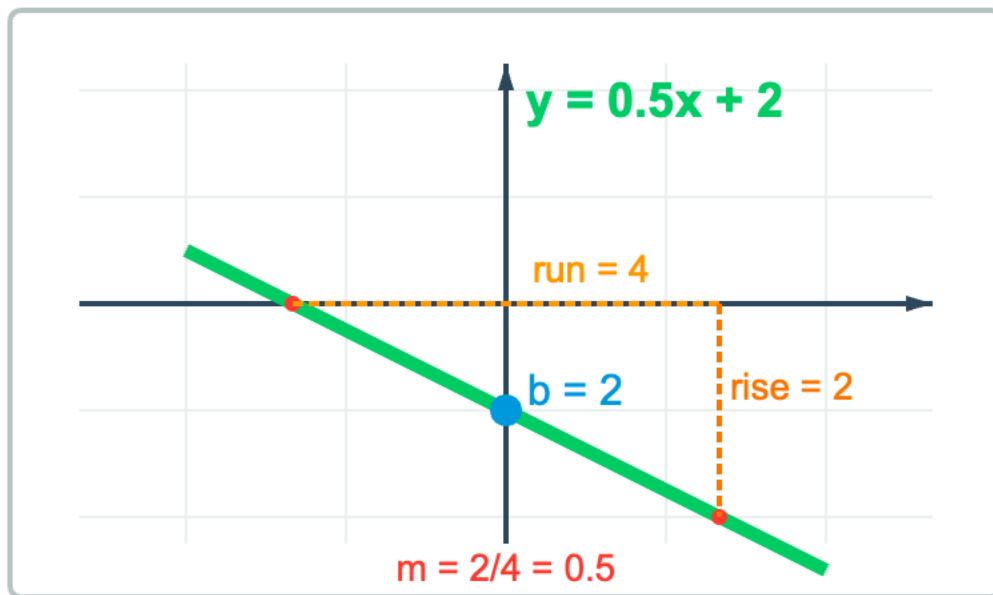


$$y = mx + b$$

Linear Equations & Machine Learning

m = slope

b = y-intercept



Linear Regression

Finds best m & b to predict values!

Figure 14.8 — Linear Equations and Machine Learning Connection. The fundamental linear equation $y = mx + b$ forms the foundation of linear regression in machine learning. This diagram illustrates how the slope ($m = 0.5$) and y—intercept ($b = 2$) define a line's behavior, with linear regression algorithms finding optimal values for these parameters to make predictions from data.

Linear Equations: $y = mx + b$

The Foundation of Linear Regression

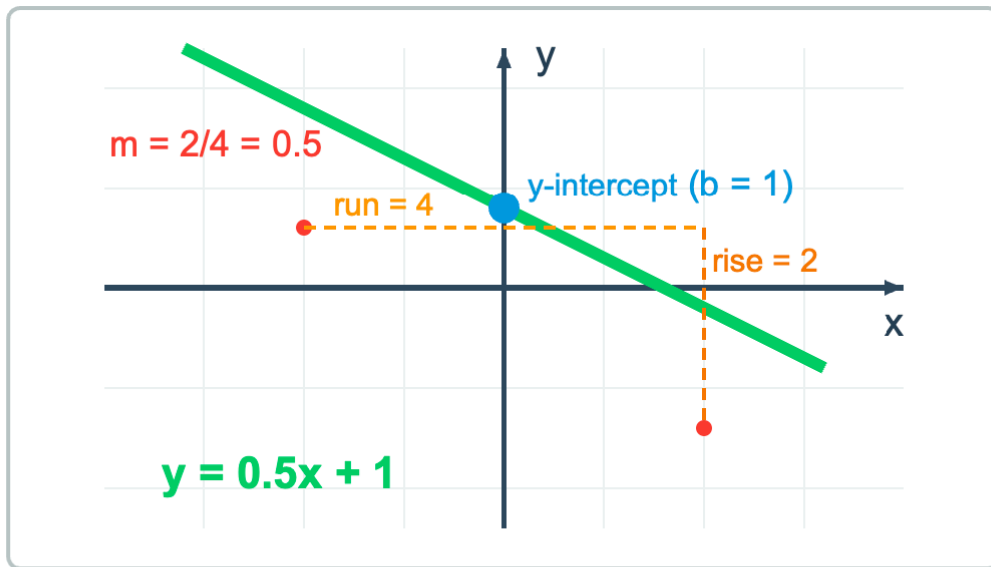
$$y = mx + b$$

m = slope

(steepness)

b = y-intercept

(crosses y-axis)



Machine Learning Connection

Linear Regression finds the best m and b to predict future values!

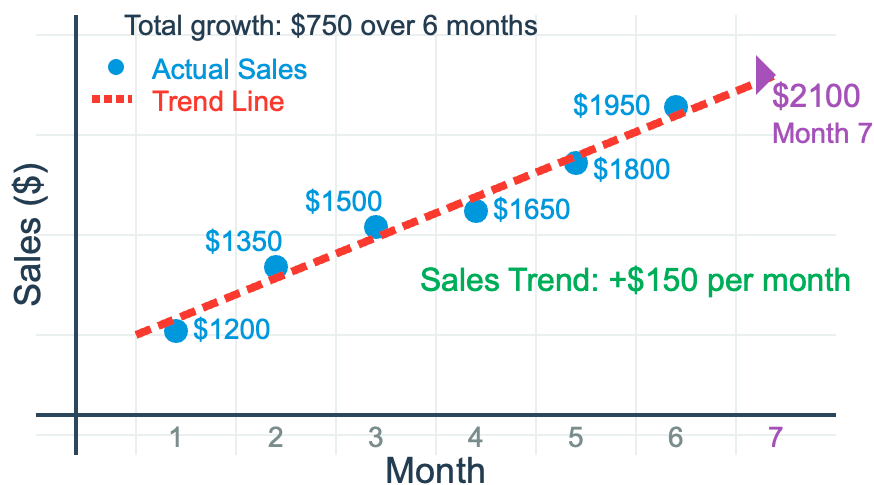
Figure 14.9 — The Linear Equation $y = mx + b$ and Its Role in Machine Learning. This diagram illustrates the fundamental components of linear equations, including slope (m) as the measure of steepness and y—intercept (b) as the point where the line crosses the y—axis. The example shows $y = 0.5x + 1$ with detailed slope calculation using rise over run ($2/4 = 0.5$). These mathematical foundations directly translate to linear regression in machine learning, where algorithms optimize these parameters to find the best—fitting line for predictive modeling.

Another Sales Trend Analysis

Linear Regression with Python

```
from scipy.stats import linregress  
sales = [1200, 1350, 1500, 1650, 1800, 1950]  
slope, intercept = linregress(months, sales)  
plt.scatter(months, sales, color= 'blue' )
```

Sales Trend Analysis



Linear Regression: slope = 150, intercept = 1050

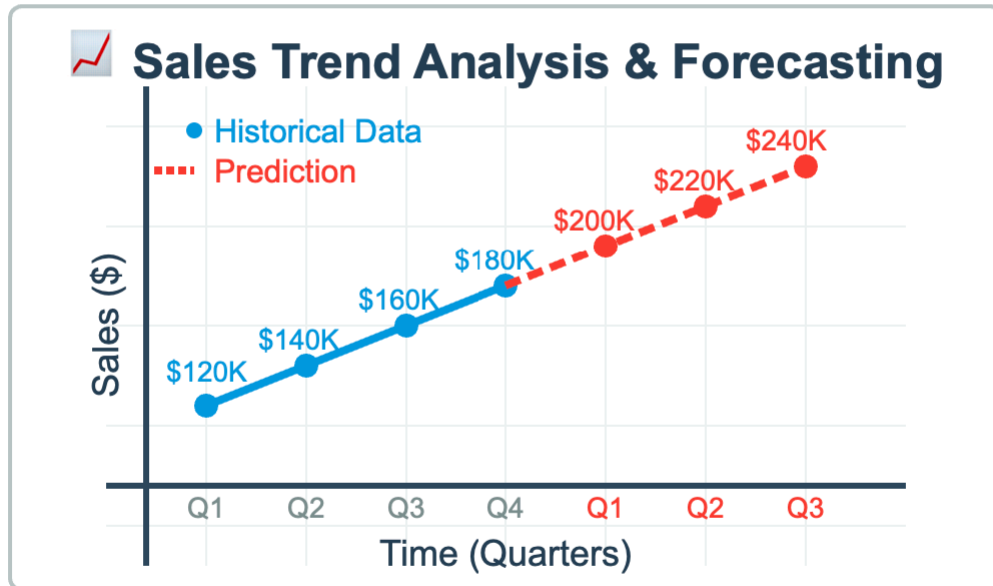
Prediction: Month 7 sales = \$2,100 (continuing \$150/month growth)

Figure 14.10 — Sales Trend Analysis Using Linear Regression in Python. This figure demonstrates a practical application of linear regression for sales forecasting, showing Python code implementation using `scipy.stats` and `matplotlib`. The scatter plot displays six months of actual sales data (\$1,200 to \$1,950) with a fitted trend line revealing consistent growth of \$150 per month. The linear regression model (slope = 150, intercept = 1,050) successfully predicts Month 7 sales at \$2,100, illustrating how mathematical models can extract meaningful patterns from business data for predictive analytics.

Real-World Applications

Using Math to Solve Problems

Coordinate geometry powers real technologies and data analysis!
From business forecasting to GPS navigation



Linear Equation: Sales = 20,000 × Quarter + 100,000

Slope (m) = \$20K growth per quarter • Y-intercept (b) = \$100K baseline

 *This math powers business intelligence, GPS, machine learning & more!*

Figure 14.11 — Real—World Applications of Linear Equations in Business Forecasting. This figure demonstrates how coordinate geometry and linear regression translate into practical business intelligence. The chart shows quarterly sales data from \$120K to \$180K with a clear upward trend, using the linear equation $\text{Sales} = 20,000 \times \text{Quarter} + 100,000$ to predict future performance reaching \$240K. The visualization illustrates how mathematical concepts power real—world technologies including business forecasting, GPS navigation, and machine learning applications, with a consistent growth rate of \$20K per quarter from a \$100K baseline.

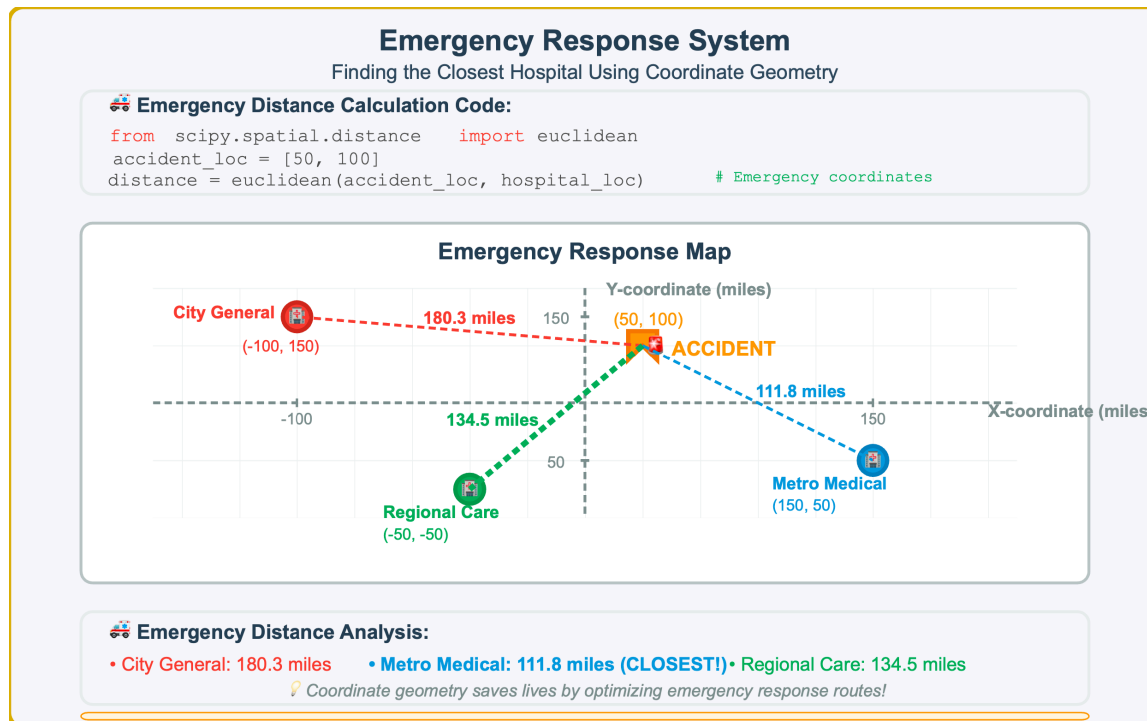
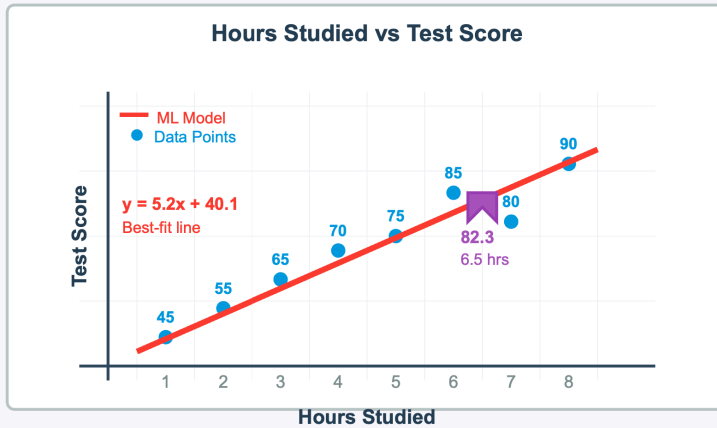


Figure 14.12 — Emergency Response System — Finding the Closest Hospital. This practical example demonstrates how coordinate geometry and distance calculations are used in emergency services to find the nearest hospital to an accident at location (50, 100). Using the euclidean distance formula, Metro Medical at 111.8 miles is determined to be the closest facility.

Machine Learning Preview: Linear Regression

Finding the Best Line Through Data Points

```
from scipy.stats import linregress
study_hours = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
slope, intercept = linregress(hours, scores)
```



Key Insight

ML Prediction:
6.5 hours → 82.3 points

How it works:

- Find best line through data points
- Use line equation to predict new values
- Coordinate geometry enables ML!

Pattern: $y = 5.2x + 40.1$

Every extra hour of study
adds ~5.2 points to score

Figure 14.13 — Machine Learning Preview — Linear Regression in Action. This example demonstrates how coordinate geometry principles enable machine learning predictions. By finding the best—fit line through study hour and test score data points, the algorithm can predict that 6.5 hours of study should yield approximately 82.3 points.

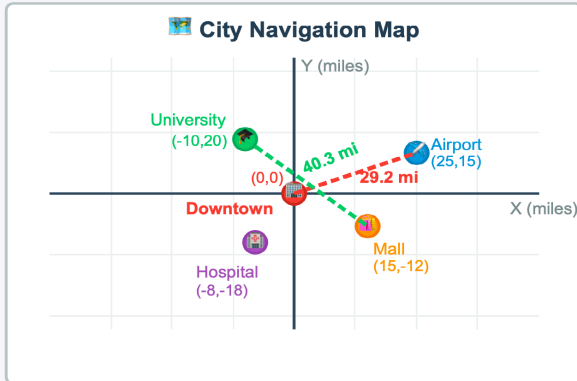
GPS Navigation System Challenge

Try It Yourself - Build a City Distance Calculator

Challenge: Build a City Distance Calculator

- Plot 5 cities on coordinate plane
- Calculate distances between any two cities
- Find shortest routes • Create trip planner for multiple stops

Use: pandas DataFrame, matplotlib plotting, scipy.spatial.distance



Your Tasks:

1. Plot Cities

Use coordinates to place all 5 cities

2. Input Two Cities

Get user input for start and destination

3. Calculate Distance

Use distance formula: $\sqrt{(x_2-x_1)^2 + (y_2-y_1)^2}$

4. Find Shortest Routes

Compare all possible paths

5. Trip Planner

Plan multi-stop journeys efficiently

Tools to Use:

- pandas DataFrame for city data
- matplotlib for plotting
- scipy.spatial.distance for calculations

Figure 14.14 — GPS Navigation System Challenge. This hands—on challenge demonstrates how coordinate geometry powers GPS navigation systems. Students plot five city locations and use the distance formula to calculate routes, mimicking real—world navigation algorithms used in mapping applications.

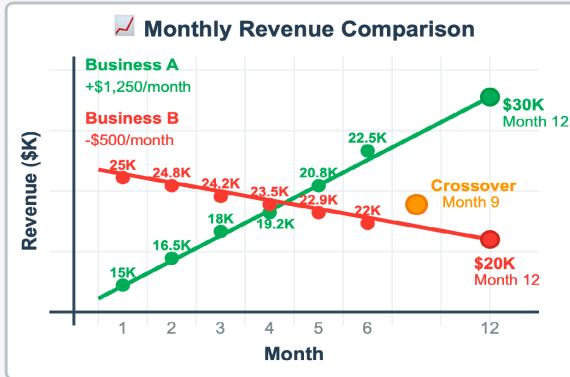
Business Growth Analyzer Challenge

Try It Yourself - Analyze Business Trends Using Slope

Challenge: Analyze Business Trends Using Slope

- Compare revenue trends for two businesses over 6 months
- Calculate growth rates (slope) • Predict future revenue
- Find crossover point

Use: pandas, matplotlib, scipy.stats.linregress for trend analysis



Your Tasks:

1. Create DataFrame

Input monthly revenue data for both businesses

2. Plot Trends

Use matplotlib to visualize growth patterns

3. Calculate Slopes

Find growth rates using scipy.stats.linregress

4. Predict Month 12

Extrapolate trends to forecast future revenue

5. Find Crossover Point

Determine when Business A overtakes Business B

Key Insight:

- Slope = growth rate per month
- Positive slope = increasing revenue
- Negative slope = declining revenue

Figure 14.15 — Business Growth Analyzer Challenge. This hands—on challenge demonstrates how slope calculations can analyze real business trends. Students compare two companies' revenue patterns over 6 months, calculate growth rates, and predict future performance using coordinate geometry principles.

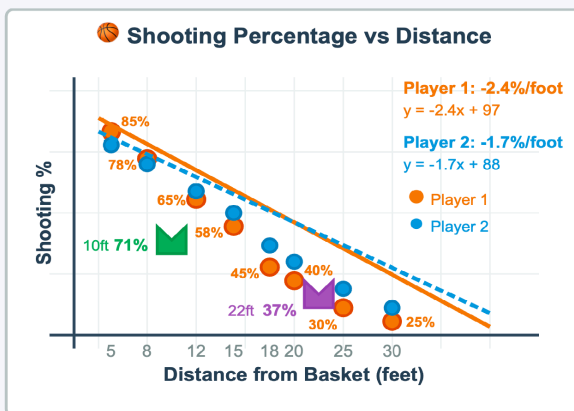
Sports Analytics Challenge

Basketball Shooting Distance Analysis

Challenge: Find Optimal Basketball Shooting Distances

- Analyze shooting percentage vs distance from basket (5-30 feet)
- Find linear relationship • Predict shooting accuracy • Compare players

Use: scatter plots, linregress for trend analysis, performance predictions



Your Tasks:

- 1. Create Scatter Plot**
Plot shooting % vs distance for both players
- 2. Find Best-Fit Line**
Use linregress to calculate trend lines
- 3. Interpret Slope**
Analyze decline rate per foot of distance
- 4. Predict Accuracy**
Use equations to forecast performance
- 5. Compare Players**
Which player is better at long-range shots?

Key Insight:

- Negative slope = accuracy decreases with distance
- Shallower slope = better long-range shooter

Figure 14.16 — Sports Analytics Challenge — Basketball Shooting Analysis. This hands-on challenge demonstrates how linear regression analyzes sports performance. Students examine shooting percentage versus distance data for two players, calculate decline rates using slope, and make performance predictions based on coordinate geometry principles.